

Digital Carbon Auditor – Problem Understanding and Scope

1. Summary in Own Words

Digital storage and compute are a substantial source of carbon emissions, but users and organizations often don't see the environmental impact of redundant, duplicate, or unused data. The system should:

1. Scan storage systems
1. Estimate carbon footprint based on stored data volume
1. Find byte-for-byte duplicates using cryptographic hashes
1. Categorize files by type and usage
1. Flag wasteful or redundant data
1. Produce reports and recommendations to cut carbon footprint

In short: make hidden carbon costs of digital data visible and suggest how to reduce them.

2. Functional vs Non-Functional Requirements

Functional Requirements

#	Requirement	Source
---	-------------	--------

FR-1	Scan digital storage systems	Core requirements
FR-2	Estimate carbon footprint from storage size	Core requirements
FR-3	Find exact duplicate files via cryptographic hash comparison	Core requirements
FR-4	Categorize files by type	Core requirements
FR-5	Categorize files by usage patterns	Core requirements
FR-6	Identify wasteful or redundant data	Core requirements
FR-7	Generate a report	Core requirements
FR-8	Include reduction recommendations in the report	Core requirements

FR-9	Include optimization recommendations in the report	Core requirements
-------------	---	--------------------------

Non-Functional Requirements (Implied)

#	Requirement	Rationale
NFR-1	Reasonable performance on large storage volumes	Large datasets, hash computation, disk I/O
NFR-2	Correct and reliable hash-based duplicate detection	Trust in results
NFR-3	Support for typical local/network file systems	Need to access real storage
NFR-4	Clear, usable reports	Users must act on findings
NFR-5	Scalability for organizational data sizes	30–50% of storage may be redundant
NFR-6	Safe, read-only scanning where possible	Do not alter or delete files

3. Explicit vs Optional / Out-of-Scope

Explicitly Required

- Scanning digital storage
- Carbon footprint estimation from storage size
- Exact duplicate detection using cryptographic hashes
- Categorization by file type and usage patterns
- Identifying wasteful or redundant data
- Report with reduction and optimization recommendations

Optional / Ambiguous (Design Choices)

- Scope of “digital storage systems” – local disk only, network shares, cloud APIs, etc.
- Definition of “usage patterns” – last access time, frequency, application metadata, or custom rules
- Carbon estimation model – CO2 per TB-hour, PUE, regional grid factors, etc.
- Report format – CLI output, HTML, PDF, JSON, etc.
- Implementation of recommendations – reporting only vs built-in cleanup/deduplication

Likely Out of Scope

- Near-duplicate detection (e.g., perceptual hashing for images) – requirement is “exact” duplicates
 - Real-time monitoring – focus on scan-and-report
 - Cloud provider APIs for live metrics – not mentioned
 - Automatic deletion or modification of files – only “recommendations”
 - User authentication or multi-tenancy – not mentioned
 - Historical trending or time-series analysis – not in requirements
-

4. Technical Constraints (Especially Hash-Based Duplicate Detection)

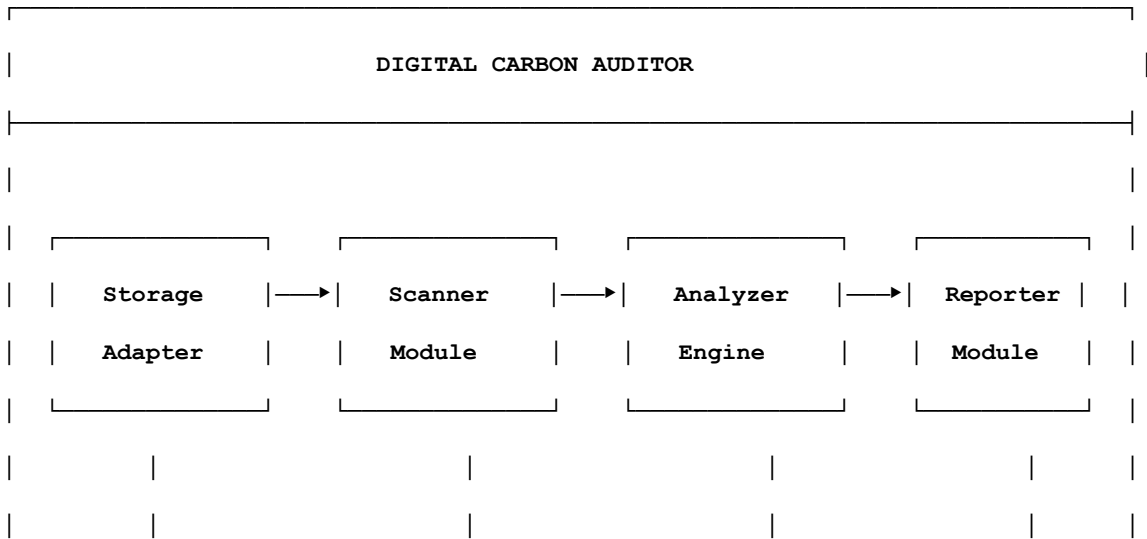
Hash-Based Duplicate Detection

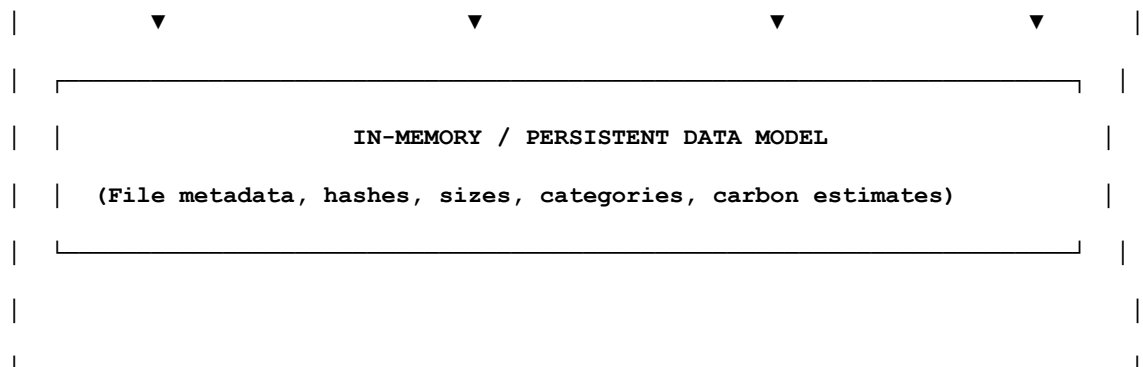
- Exact duplicates only: detection is via cryptographic hashes (e.g., SHA-256), not similarity or perceptual hashing.
- Collision resistance: use a strong hash (SHA-256 or similar) so different files are practically never mapped to the same hash.
- Read and I/O cost: full content must be read to compute hashes; big files and large volumes are I/O-intensive.
- Ordering strategy:
 - Common optimizations: hash only after filtering (e.g., by size, or by fast hash), or use a two-phase approach (fast hash first, then full hash for candidates).
 - Problem statement requires “cryptographic hash comparison” but does not forbid a multi-stage strategy to reduce work.
- Equality semantics: files with the same hash are treated as exact duplicates; name, path, timestamps do not matter.

Other Constraints

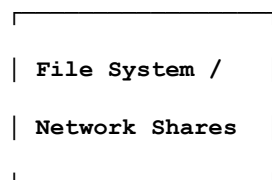
- Read-only scanning (implied): no automatic modifications to stored data.
- Storage size dependency: carbon estimation depends on storage size, so size must be available for each object.
- File type and usage: categorization needs metadata (extension, MIME type, timestamps, etc.) and possibly OS APIs for usage.

5. High-Level System Architecture





EXTERNAL BOUNDARIES



Components

1. Storage Adapter
 - Connects to local and/or remote storage
 - Walks directory trees
 - Exposes a unified interface for path, size, timestamps, and stream access
 - Abstracts OS and filesystem details
1. Scanner Module
 - Iterates over files via the adapter
 - Extracts metadata (path, size, timestamps, extension, MIME)
 - Computes cryptographic hashes (with optional optimization such as size pre-filter or fast-hash phase)
 - Feeds the analyzer with metadata and hashes
1. Analyzer Engine
 - Duplicate detection: groups files by hash to find exact duplicates
 - Carbon estimation: applies chosen model (storage size → energy → CO2)
 - Categorization: groups files by type (and optionally usage patterns)
 - Waste/redundancy: flags duplicates, ROT, and possibly low-value categories
1. Reporter Module
 - Builds reports from analysis results
 - Includes reduction and optimization recommendations
 - Supports at least one output format (CLI/HTML/PDF/JSON, etc.)

1. Data Model

- In-memory during scan; optionally persisted
- Stores file metadata, hashes, sizes, categories, carbon estimates
- Supports aggregation and grouping for reports

Data Flow

Storage → Scan (metadata + hash) → Analyze (duplicates, carbon, categories, waste)
→ Report (findings + recommendations)

6. Assumptions and Limitations

Assumptions

- Storage is accessible via standard file APIs (local or mounted)
- Carbon footprint estimation uses a model based on storage size (e.g., kWh per TB-year, grid factor)
- “Usage patterns” can be approximated from OS metadata (access/modification times, etc.)
- Users/organizations will not run automatic destructive actions; recommendations are advisory
- Cryptographic hash comparison is acceptable for exact-duplicate detection; near-duplicates are out of scope
- There is no requirement for real-time or continuous monitoring in the initial version

Limitations

- Estimation only: carbon numbers are estimates, not measurements
 - Exact duplicates only: no near-duplicate or semantic similarity detection
 - Storage size proxy: energy/carbon are modeled from size, not measured
 - Data center mix: model may use average industry data; regional and provider differences may be ignored
 - Access pattern: current design is scan-based, not continuous monitoring
 - Scope: depends on what storage systems the adapter supports (e.g., local vs cloud APIs)
-

Confirmation

I understand:

1. Exact duplicate detection via cryptographic hashes is mandatory.
1. Carbon estimation is derived from storage size, not direct energy measurement.
1. Reports must contain actionable reduction and optimization recommendations.
1. Scope of storage systems, usage patterns, carbon model, and report format are design choices to be fixed before implementation.

I will not write implementation code until you confirm this understanding or request changes.

Digital Carbon Auditor – USPs and Non-Functional Excellence

1. Unique Selling Points (USPs)

USP 1: Carbon Storytelling, Not Just Numbers

Differentiator: Move beyond raw bytes and CO2 figures. Turn findings into clear, contextual narratives users can act on and share. Ideas:

- “Equivalence” language: e.g. “X GB = N tree-months of sequestration” or “Same energy as Y smartphone charges”
- “What-if” views: “If you removed duplicates alone, you’d save Z kg CO2e per year”
- Per-user or per-department “digital carbon budget” views for organizations
- “Carbon journey” for a file or folder: storage today vs if deduplicated and archived

Why it matters: Most tools report numbers without helping people relate to them.

USP 2: Tiered Duplicate Detection with Clear Confidence Levels

Differentiator: Transparent, layered duplicate analysis instead of a single yes/no. Ideas:

- Tier 1 – Exact duplicates (100%): SHA-256 match
- Tier 2 – Near-duplicates (configurable): Optional second pass for images/documents with perceptual or content hashing
- Tier 3 – Structural redundancy: Same content under different names/formats (e.g. PDF vs DOCX of same report)
- Each tier has a clear “confidence” label and estimated savings range
- User chooses which tiers to run (speed vs depth)

Why it matters: Gives users confidence in what “duplicate” means and how conservative or aggressive the system is.

USP 3: Carbon Model Transparency and Tunability

Differentiator: Users see and adjust the model assumptions behind carbon estimates. Ideas:

- Explicit parameters: regional grid factor, PUE, storage efficiency, server type
- Presets: “US average”, “EU (cleaner grid)”, “hyperscaler cloud”
- “Sensitivity” view: e.g. how estimates change if grid gets 20% cleaner
- Option to plug in custom factors or local data center data
- Clear audit trail: which factors were used for each estimate

Why it matters: Builds trust and supports varied organizational contexts and sustainability goals.

USP 4: “Digital Waste Taxonomy” – Categorization That Drives Action

Differentiator: Categories designed around *what to do next*, not just file type. Ideas:

- ROT: Redundant, Obsolete, Trivial
- “Orphaned” data: no recent access, low value
- “High-carbon, low-value”: large, rarely used, easy to archive
- “Duplicate clusters”: sets of identical files with suggested “keep one, remove others” options
- “High-risk” categories: e.g. sensitive data in wasteful locations
- Each category linked to specific recommendations (delete, archive, deduplicate, compress, move)

Why it matters: Turns generic categorization into a decision framework.

USP 5: Organizational Learning and Gamification (Optional)

Differentiator: Make reduction a team effort and track progress. Ideas:

- “Digital carbon score” per team/department
- Simple leaderboards (with privacy controls)
- Badges/milestones: “First 10% reduction”, “Duplicate-free team”
- Short “digital waste scorecards” suitable for sustainability reports or ESG dashboards
- Optional integration with existing sustainability or IT dashboards

Why it matters: Helps sustainability and IT initiatives gain traction inside organizations.

USP 6: Privacy-First, Zero-Knowledge Design

Differentiator: Operate on sensitive data while minimizing trust and exposure. Ideas:

- Hash computation and duplicate detection on-device or in trusted boundary
- Optional “aggregate-only” mode: only totals/summaries leave the environment
- No telemetry or upload of file content
- Option to hash or anonymize paths in reports
- Clear “data residency” settings (e.g. EU-only processing)

Why it matters: Essential for adoption in regulated industries (finance, healthcare, legal).

USP 7: Multi-Modal Storage Awareness

Differentiator: Different storage tiers modeled with different carbon profiles. Ideas:

- Hot vs cold storage (SSD vs HDD vs tape vs cloud tiers)
- Different carbon intensity by tier and region
- Recommendations like “Move to cold/archive” with estimated impact
- Awareness of cloud provider sustainability commitments (e.g. renewable targets)
- “Right-tiering” recommendations: where to place data for lowest carbon

Why it matters: Most tools treat all storage the same; reality is very different across tiers and clouds.

2. Non-Functional Requirements – Deep Dive

NFR 1: Performance at Scale

Aspect	Approach	Rationale
Incremental scanning	Remember prior scans; only hash new/changed files	Avoid re-processing large volumes on each run
Parallel I/O	Hash files in parallel with bounded concurrency	Use available I/O and CPU
Size pre-filter	Only hash files with same size (duplicates must match)	Cut hash work dramatically
Fast-hash first	Use xxHash/Murmur for candidate set, SHA-256 for final match	Fewer heavy hashes per file
Streaming hashes	Hash while reading, avoid buffering whole file	Lower memory use for big files
Configurable scope	Exclude paths, file types, sizes	Focus scans where it matters

Progress and cancellation	Visible progress, safe interrupt and resume	Usability on long scans
---------------------------	---	-------------------------

NFR 2: Reliability and Correctness

Aspect	Approach	Rationale
Hash algorithm	SHA-256 or SHA3-256	Industry standard, collision-resistant
Verification	Optional re-check of duplicates before recommending deletion	Safety for destructive actions
Error handling	Graceful handling of unreadable, locked, or changed files	Avoid failing entire scan
Integrity	Checksums on scan results for auditability	Reproducible reports
Determinism	Same inputs produce same outputs	Repeatable audits

NFR 3: Usability and Accessibility

Aspect	Approach	Rationale
Outputs	CLI, JSON, HTML, PDF	Different users and workflows
Progressive detail	Summary → drill-down (folder, category, duplicates)	Avoid overwhelming users
Defaults	Sensible defaults for all options	Works without deep configuration
Documentation	Clear explanation of metrics, models, and recommendations	Transparency and trust
Accessibility	Semantic structure, contrast, screen-reader support	Broader organizational use

NFR 4: Security

Aspect	Approach	Rationale
--------	----------	-----------

Least privilege	Read-only access; no modification of scanned data	Reduce blast radius
Credential handling	Use OS credential store; no hardcoded secrets	Secure access to remote storage
Output sanitization	Avoid exposing full paths/sensitive names in logs/reports	Compliance and privacy
Supply chain	Minimal dependencies; prefer well-maintained libraries	Lower vulnerability surface

NFR 5: Extensibility and Maintainability

Aspect	Approach	Rationale
Pluggable storage	Abstract interface for local, SMB, NFS, cloud APIs	Add new backends without rewrites
Pluggable carbon model	Strategy or plugin for different estimation models	Adapt to new research and regions
Pluggable categorizers	Extensible taxonomy and rules	Custom waste definitions
Pluggable reporters	Different output formats and integrations	Fit into existing workflows
Configuration	Config file / env vars; no code changes for tuning	Operational flexibility

NFR 6: Observability and Debugging

Aspect	Approach	Rationale
Logging	Structured logs with levels; optional file output	Debugging and support
Timing	Per-phase and per-file metrics where useful	Performance tuning
Dry-run mode	Simulate actions without changing anything	Safe testing
Verbose mode	Detailed progress and decisions	Understanding behavior

3. Innovative Feature Ideas (Beyond Core Requirements)

Feature	Description	Value
Scheduled scans	Cron or daemon; track trends over time	Shows improvement and regression
Baseline comparison	Compare current vs previous scan	Demonstrates impact of cleanup
What-if scenarios	"If I delete duplicates + archive ROT..."	Clear impact of actions
Export for sustainability reports	ESG/CSR-ready summaries	Alignment with reporting needs
Integration with dedup tools	Output suitable for external dedup/archive tools	Practical workflow
Resource caps	Limit CPU, memory, I/O during scan	Safe for production systems
Snapshot-aware scanning	Optional use of snapshots to avoid impacting live workloads	Enterprise readiness
Compliance tags	Mark retention/compliance-relevant data	Avoid recommending deletion of regulated data

4. Implementation Philosophy (Without Code)

Design Principles

1. Scan-once, analyze-many – Store hashes and metadata; run different analyses and reports without re-scanning.
1. Fail safely – Never delete or modify files; only produce recommendations.
1. Transparent by default – Make carbon model, confidence, and assumptions visible.
1. Progressive enhancement – Core works with minimal setup; advanced features opt-in.
1. Local-first, cloud-optional – Core logic runs locally; cloud only for optional features.

Suggested Priority

Phase	Focus	Outcome
Phase 1	Core: scan, hash, exact duplicates, basic carbon estimate, simple report	Usable MVP
Phase 2	Robust NFRs (performance, resilience, multi-format reports)	Production-ready
Phase 3	USPs: carbon storytelling, transparency, taxonomy	Differentiation
Phase 4	Advanced: near-duplicates, multi-storage, org features	Best-in-class

5. Summary: What Makes It “Best”

- Trust: Transparent carbon model and clear confidence levels for duplicates.
- Actionability: Waste taxonomy and recommendations tied to specific actions.
- Scale: Incremental scans, parallelism, and size pre-filter for large datasets.
- Privacy: Local processing, minimal data, optional aggregation-only exports.
- Narrative: Carbon equivalences and what-if scenarios that resonate with users.
- Flexibility: Pluggable storage, carbon models, and reporters for different contexts.
- Safety: Read-only scanning, dry-run, and optional verification before any destructive step.

Together, these form a coherent product and implementation strategy that stays within the stated requirements while aiming for a distinctive, enterprise-ready Digital Carbon Auditor.

